# Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

## 1. Datos de Identificación del Estudiante y la Práctica

| | |
|---|---|
| **Nombre del estudiante(s)** | Cael Alejandro Soto Castillo |
| **Asignatura** | Simulación |
| **Ciclo** | 5 A |
| **Unidad** | U1: Introducción a la Simulación |
| **Resultado de aprendizaje de la unidad** | R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 004 |
| **Título de la Práctica** | Análisis comparativo de herramientas de simulación mediante un modelo común |
| **Nombre del Docente** | José O. Guamán Q. |
| **Fecha** | 22 de octubre de 2026<br>23 de octubre de 2026 |
| **Horario** | Jueves 10h30 a 11h30<br>Viernes 07h30 a 09h30 |
| **Lugar** | Aula |
| **Tiempo planificado en el Sílabo** | 3 horas |

## 2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación [1].

1. Implementar un modelo sencillo (epidemiológico SIR) con las mismas reglas y parámetros en distintas herramientas.
2. Comparar la sintaxis, la facilidad de implementación y la visualización de cada herramienta.
3. Identificar el ámbito de aplicación de cada herramienta y explicar las diferencias producidas por la generación de números aleatorios.

En este trabajo se desarrollaron y ejecutaron las versiones en **Java y NetLogo**. De las tres herramientas de la ficha, a este grupo le correspondió programar la versión en Java, por lo que no se implementó Python/SimPy y no se presentan resultados experimentales de esa herramienta.

## 3. Materiales y reactivos

- Ficha de la práctica (Guía de Actividades Práctico-Experimentales Nro. 004) y código fuente del proyecto.
- Modelo SIR con población de 100 personas y estados S, I y R.

- Archivos CSV de resultados y de parámetros para pasar la información de Java a NetLogo.
- JDK 11 o posterior y NetLogo (con su extensión `csv`).

# 4. Equipos y herramientas

- Computador con sistema operativo y editor de código.
- JDK 11 o posterior para compilar y ejecutar Java (se compila con `-release 11`). Solo se usa la biblioteca estándar: no se necesita Maven, Gradle ni librerías externas.
- NetLogo 7.0.4 de escritorio [2], con la extensión `csv` que ya incluye.
- Git y GitHub para el control de versiones.
- LaTeX (Overleaf) para redactar el informe.

# 5. Procedimiento / Metodología Ejecutada

## 5.1. ¿En qué consiste el modelo SIR?

El modelo SIR sirve para representar cómo se propaga una enfermedad en un grupo de personas. Cada persona está en uno de tres estados [1]:

- **S (susceptible):** todavía no se ha contagiado y puede hacerlo.
- **I (infectada):** tiene la enfermedad y puede contagiar a otros.
- **R (recuperada):** ya se curó y no vuelve a contagiarse.

En nuestro caso nadie nace, muere ni llega de afuera, así que el total de personas no cambia:

$$N = S + I + R = 100.$$

Dos valores controlan el modelo: $\beta$, que indica qué tan fácil ocurre un contagio, y $\gamma$, que indica qué tan rápido se recupera una persona. La ficha los relaciona con tres ecuaciones que describen el paso de S a I y de I a R:

$$\frac{dS}{dt} = -\beta\frac{SI}{N}, \qquad \frac{dI}{dt} = \beta\frac{SI}{N} - \gamma I, \qquad \frac{dR}{dt} = \gamma I.$$

Con $\beta = 0{,}30$ y $\gamma = 0{,}10$, el número reproductivo básico es $\mathcal{R}_0 = \beta/\gamma = 3$: en promedio, una persona infectada contagiaría a unas tres. Es solo una referencia teórica del modelo, no una cantidad exacta que deba producir cada persona en una corrida con azar.

La ficha aclara que no hace falta resolver las ecuaciones: se puede simular paso a paso. Así lo hicimos, tratando a cada persona de forma individual.

| Parámetro de la corrida base | Valor |
|---|---|
| Población $N$ | 100 |
| Susceptibles iniciales $S(0)$ | 95 |
| Infectados iniciales $I(0)$ | 5 |
| Recuperados iniciales $R(0)$ | 0 |
| Probabilidad de contagio $\beta$ | $0{,}30$ |
| Probabilidad de recuperación $\gamma$ | $0{,}10$ |
| Tiempo de simulación | 100 pasos |
| Semilla (para repetir cada corrida) | 42 |

**Tabla 1:** Configuración común. Los valores proceden de la ficha; la semilla la añadimos nosotros.

## 5.2. Reglas que usamos en las dos herramientas

Antes de programar definimos unas reglas sencillas, iguales para Java y NetLogo, para que ambas representen exactamente el mismo modelo:

- **Contagio.** En cada paso, cada persona infectada elige al azar a un contacto entre las otras 99 personas. Si ese contacto era susceptible al comenzar el paso, se contagia con probabilidad $\beta$; es decir, más o menos 3 de cada 10 veces.
- **Recuperación.** En ese mismo paso, cada persona infectada tiene probabilidad $\gamma$ de recuperarse; es decir, más o menos 1 de cada 10 veces.
- **Cambios al mismo tiempo.** Los cambios de un paso se aplican todos juntos al final. Así, quien se acaba de contagiar empieza a contagiar recién en el paso siguiente.
- **Conteo.** Al terminar cada paso contamos cuántas personas hay en S, I y R y lo guardamos. Registramos desde $t = 0$ hasta $t = 100$, es decir, 101 registros, aunque ya no queden infectados.

Usamos la semilla 42 para poder repetir cada simulación y obtener siempre lo mismo [3]. Aun así, Java y NetLogo generan sus números al azar de forma distinta, por lo que no esperamos que den exactamente la misma evolución. Hicimos una corrida por herramienta.

```
Inicializar 95 susceptibles, 5 infectados y 0 recuperados
Guardar el estado inicial en t = 0
Para t = 1 hasta 100:
    Copiar los estados de todas las personas
    Para cada persona infectada al inicio del paso:
        Elegir otra persona como contacto
        Si el contacto es susceptible: contagiarlo con prob. β
        Recuperarse con probabilidad γ
    Aplicar los cambios al mismo tiempo
    Contar y guardar S, I y R
```

## 5.3. Pasos que realizamos

La ficha pedía el modelo en tres herramientas; a nuestro grupo le correspondió Java, y usamos NetLogo para visualizar. No hicimos la versión en Python/SimPy. Estos fueron los pasos:

1. **Analizamos la ficha y las fórmulas.** Revisamos las ecuaciones del modelo SIR, el número reproductivo y los parámetros que debíamos usar (Tabla 1).
2. **Definimos las reglas.** Acordamos las reglas de la sección anterior para que ambas herramientas fueran comparables.
3. **Elegimos cómo organizar el programa.** Decidimos seguir el esquema **modelo–vista–controlador** en Java, para que cada parte tuviera una sola tarea:
   - el *modelo* (`Persona` y `SimulacionSIR`) guarda las personas, sus estados y las reglas de la simulación;
   - la *vista* (`VistaConsola`) muestra los parámetros y los resultados en la terminal;
   - el *controlador* (`SimulacionControlador`) ejecuta el modelo y guarda los archivos;
   - `Main` inicia el programa y avisa si hay algún error al guardar.
4. **Programamos y ejecutamos Java.** Desde la carpeta del proyecto compilamos y ejecutamos con estos dos comandos:

```
javac -encoding UTF-8 --release 11 -d build \
  src/Main.java src/modelo/*.java \
  src/vista/*.java src/controlador/*.java
java -cp build Main
```

Java muestra una tabla cada diez pasos, el máximo de infectados, el paso en que ocurre y lo que tardó el cálculo. Antes de simular revisa que los parámetros sean válidos (por ejemplo, que las probabilidades estén entre 0 y 1); si no lo son, avisa y no reemplaza los archivos anteriores.

5. **Guardamos los datos en archivos CSV.** Un CSV es un archivo de texto con datos separados por comas, que otras herramientas pueden leer fácilmente [4]. Java crea dos: `resultados_java.csv`, con la cantidad de S, I y R en cada uno de los 101 registros, y `parametros_java.csv`, con los parámetros usados (población, infectados iniciales, $\beta$, $\gamma$, tiempo máximo y semilla). Los guarda en la carpeta `resultados` y deja una copia idéntica en la carpeta `netlogo`.

6. **Construimos el modelo en NetLogo.** Abrimos `modelo_sir_netlogo.nlogox` (el formato de NetLogo 7). NetLogo cumple dos funciones: simula por su cuenta con las mismas reglas y muestra la serie de Java sin modificarla. Sus elementos principales son:

   - **Personas:** figuras que cambian de color: azul si son susceptibles, rojas si están infectadas y verdes si se recuperaron. La etiqueta **P0** marca al paciente cero, una de las cinco personas infectadas al inicio.
   - **ACTUALIZAR JAVA:** lee los archivos de Java, reinicia el escenario y muestra los parámetros activos. No compila ni ejecuta Java: eso se hace antes en la terminal. El botón **setup** carga los mismos archivos.
   - **go:** avanza la simulación propia de NetLogo hasta $t = 100$; si se pulsa de nuevo, se detiene.
   - **PAUSA:** deslizador de 0 a 0,50 segundos que hace la animación más lenta o rápida sin cambiar los resultados.
   - **Cinco gráficas:** *Evolucion Java*, *Evolucion NetLogo* y la comparación de susceptibles, infectados y recuperados (Java en negro y NetLogo en naranja). La curva de Java se carga completa desde el inicio; la de NetLogo suma un punto por paso.

7. **Hicimos la corrida base.** Ejecutamos las dos herramientas con los parámetros de la Tabla 1 y guardamos los datos.

8. **Elaboramos las gráficas del informe.** A partir de los datos de las dos herramientas preparamos las figuras que aparecen en este informe (Tabla 2).

9. **Probamos un cambio de parámetros.** Repetimos la simulación con $\gamma = 0{,}40$ para comprobar que los datos nuevos de Java llegaban bien a NetLogo y ver el efecto de una recuperación más rápida. Para repetirlo basta con cambiar el valor en `SimulacionSIR.java`, compilar y ejecutar Java, pulsar ACTUALIZAR JAVA en NetLogo y luego **go**. El resto del trabajo se mantuvo con $\gamma = 0{,}10$.

10. **Verificamos el funcionamiento.** Comprobamos que:

    - en cada registro $S + I + R = 100$, $S$ nunca aumenta y $R$ nunca disminuye;
    - Java genera 101 registros y las copias de los archivos son idénticas;
    - las cinco gráficas de NetLogo tienen los puntos esperados y, al volver a cargar los datos, no se duplican;

- al pulsar **setup**, NetLogo repite la misma ejecución con la misma semilla, y **go** se detiene al llegar a $t = 100$.

### 5.4. Gráficas del informe

Todas las gráficas tienen en el eje horizontal el *Tiempo (pasos)* y en el vertical el *Número de personas*. En las gráficas de cada herramienta, $S$ va en azul, $I$ en rojo y $R$ en verde (los mismos colores de las figuras de NetLogo); en las comparaciones, Java va en negro y NetLogo en naranja.

| Fig. | Archivo (en `figures/`) | Contenido |
|---|---|---|
| 1 | `01_evolucion_java` | $S(t)$, $I(t)$ y $R(t)$ de la corrida de Java. |
| 2 | `02_evolucion_netlogo` | $S(t)$, $I(t)$ y $R(t)$ de la simulación propia de NetLogo. |
| 3 | `03_comparacion_susceptibles` | $S(t)$ de Java frente a NetLogo. |
| 4 | `04_comparacion_infectados` | $I(t)$ de Java frente a NetLogo. |
| 5 | `05_comparacion_recuperados` | $R(t)$ de Java frente a NetLogo. |
| 6 | `06_efecto_gamma` | $I(t)$ con $\gamma = 0{,}10$ y $\gamma = 0{,}40$, un panel por herramienta. |

**Tabla 2:** Figuras del informe, con el nombre del archivo que debe tener cada una.

## 6. Resultados esperados

Según la Guía APE 004, al finalizar la práctica se espera disponer de [1]:

- Un modelo SIR implementado en NetLogo.
- Un modelo equivalente en Python mediante SimPy [5] y otro en Java.
- Archivos CSV generados por Python y Java.
- Gráficas de evolución de susceptibles, infectados y recuperados.
- Una comparación de los resultados obtenidos.
- Una visualización en NetLogo con los datos generados por Python y Java.
- Un análisis de las ventajas y limitaciones de cada herramienta, con énfasis en las diferencias producidas por la generación de números aleatorios.

Con el trabajo realizado por mi grupo logré obtener las versiones en Java y NetLogo, el CSV de Java, la visualización de su serie en NetLogo, las gráficas de comparación y una prueba de cambio de parámetros. Lo relacionado con Python/SimPy no me correspondió.

# 7. Resultados

## 7.1. Resumen de la corrida base

| Medida | Java | NetLogo |
|---|---|---|
| Registros de tiempo | 101 | 101 |
| Máximo de infectados | 29 | 44 |
| Primer tiempo del máximo | 31 | 11 |
| Primer tiempo con $I = 0$ | 77 | 55 |
| Susceptibles en $t = 100$ | 13 | 8 |
| Infectados en $t = 100$ | 0 | 0 |
| Recuperados en $t = 100$ | 87 | 92 |
| Población total conservada | 100 | 100 |

**Tabla 3:** Resultados de una corrida por herramienta: $N = 100$, $\beta = 0{,}30$, $\gamma = 0{,}10$ y semilla 42.

## 7.2. Gráficas de la corrida base

Las Figuras 1 a 5 muestran la evolución de $S$, $I$ y $R$ por herramienta y su comparación directa.

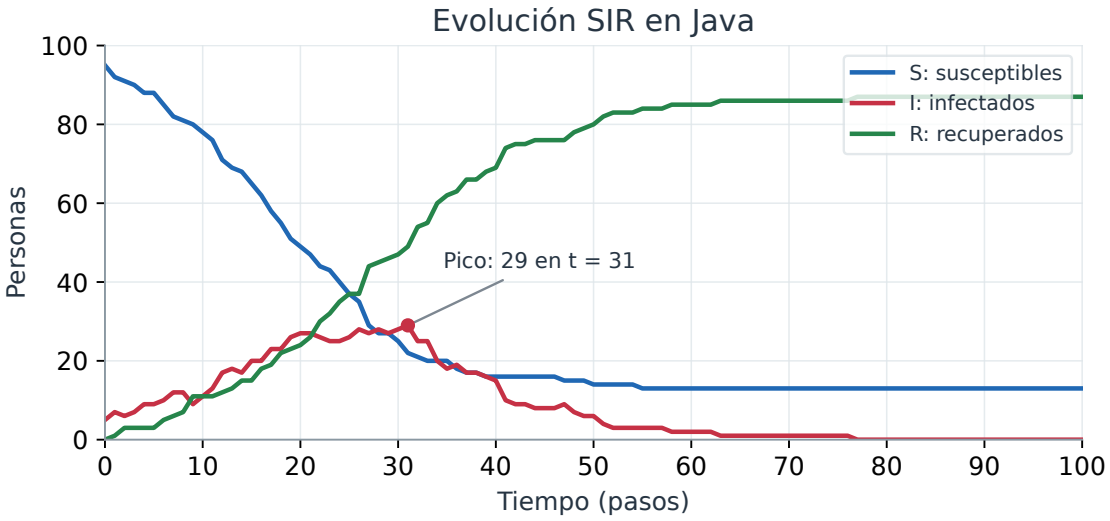



**Figura 1:** Evolución SIR obtenida por Java ($N = 100$, $\beta = 0{,}30$, $\gamma = 0{,}10$, semilla 42). El máximo de infectados es 29 en $t = 31$; en $t = 100$ quedan 13 susceptibles y 87 recuperados.

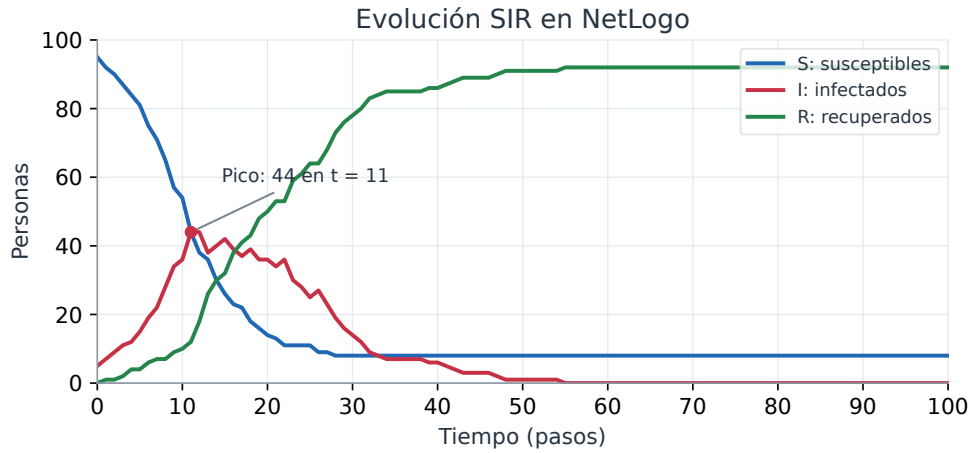



**Figura 2:** Evolución SIR calculada por NetLogo con los mismos parámetros. El primer máximo de infectados es 44 en $t = 11$; al final quedan 8 susceptibles y 92 recuperados.

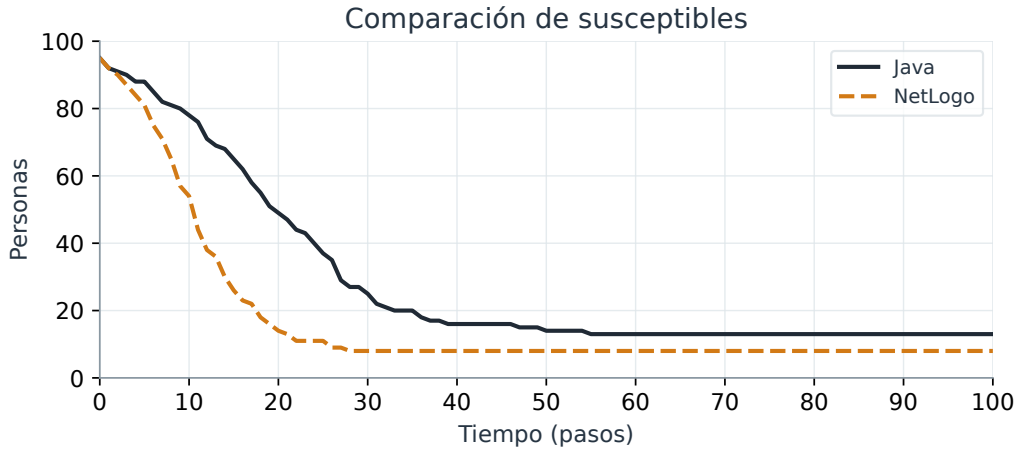



**Figura 3:** Comparación de susceptibles $S(t)$. Ambas series comienzan en 95 y terminan en 13 (Java, negro) y 8 (NetLogo, naranja).

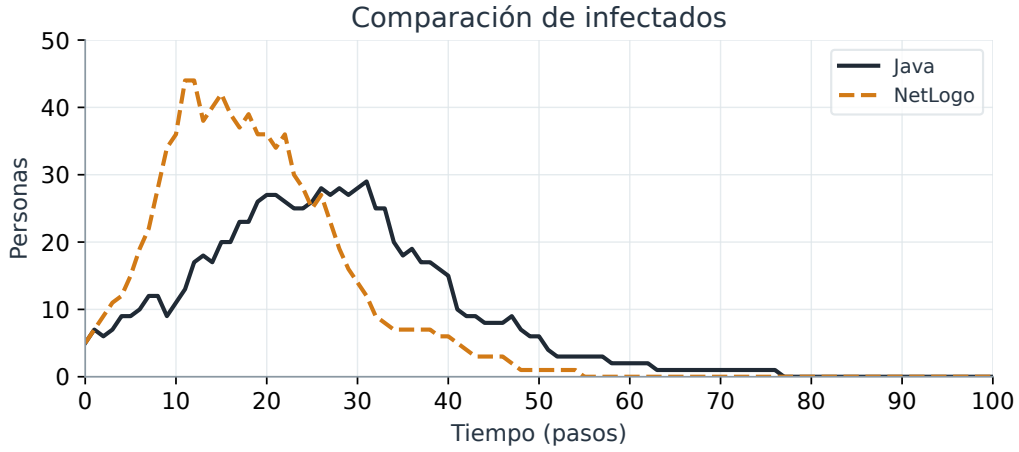



**Figura 4:** Comparación de infectados $I(t)$. Java alcanza su pico en $t = 31$ y NetLogo en $t = 11$; los infectados llegan a cero en $t = 77$ y $t = 55$, respectivamente.

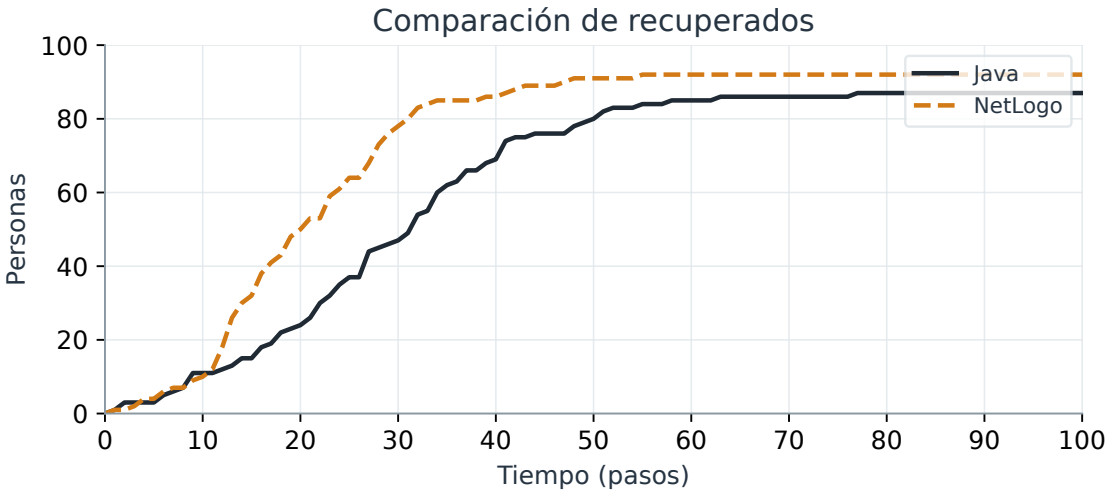



**Figura 5:** Comparación de recuperados $R(t)$. Al final se registran 87 recuperados en Java y 92 en NetLogo.

## 7.3. Efecto de cambiar $\gamma$

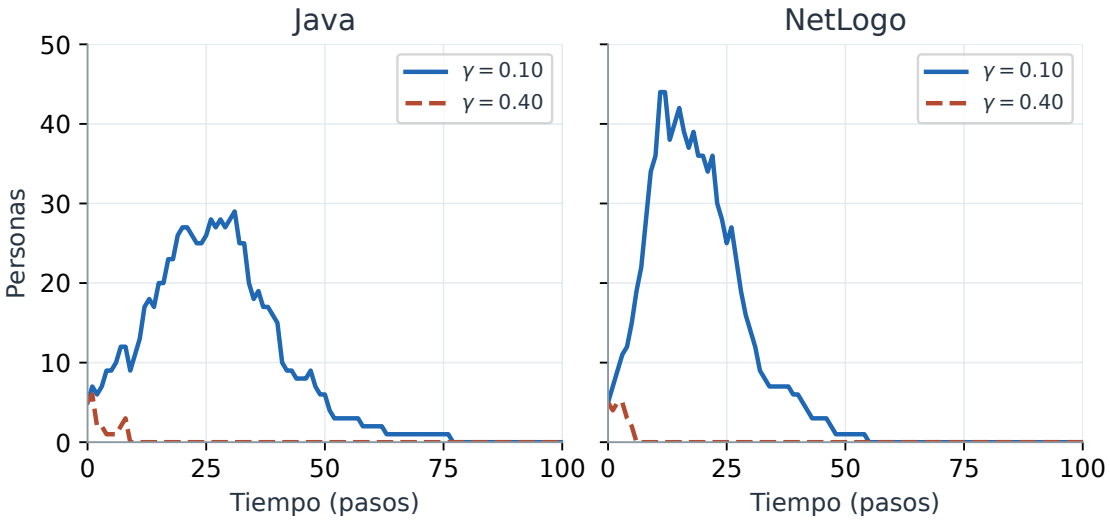



**Figura 6:** Efecto de cambiar $\gamma$ de $0{,}10$ a $0{,}40$ manteniendo los demás parámetros. Cada panel corresponde a una implementación independiente.

| Medida con $\gamma = 0{,}40$ | Java | NetLogo |
|---|---|---|
| Pico de infectados | 6 | 5 |
| Primer tiempo del pico | 1 | 2 |
| Primer tiempo sin infectados | 9 | 6 |
| Estado final $(S, I, R)$ | $(89, 0, 11)$ | $(89, 0, 11)$ |

**Tabla 4:** Corridas de demostración con $\gamma = 0{,}40$ (mismos $N$, $\beta$, horizonte y semilla).

## 7.4. Comparación de la experiencia de implementación

**Tabla 5:** Comparación basada en la experiencia obtenida durante la práctica. Los tiempos de cada herramienta abarcan tareas distintas; no se efectuó una comparación controlada de rendimiento. La columna Python/SimPy no se completa porque esa herramienta no fue asignada al grupo.

| Característica | NetLogo | Python/Sim | Java |
|---|---|---|---|
| Facilidad de implementación | Fue más cómodo, porque las reglas, las personas y las gráficas se arman dentro del mismo programa. | No asignado. | Tomó más trabajo: hubo que crear las clases, manejar los arreglos y programar cómo se guardan los archivos. |
| Manejo de agentes | Cada persona es un agente con su propio estado y color. | No asignado. | Cada persona es un objeto de la clase Persona, con un estado que puede ser S, I o R. |
| Generación de datos | Calcula su propia serie y también puede leer los CSV que genera Java. | No asignado. | Calcula la serie y la guarda en dos CSV: uno con los resultados y otro con los parámetros. |
| Visualización | Se ve a las personas cambiar de color, y las gráficas y monitores se actualizan en pantalla. | No asignado. | Solo muestra tablas en la terminal; las curvas las vimos en NetLogo. |
| Flexibilidad | Se puede cambiar una regla y ver enseguida cómo reaccionan las personas. | No asignado. | Al dividir el programa en modelo, vista y controlador, se puede cambiar una parte sin tocar las demás. |
| Tiempo de ejecución | 0.006 s | No asignado. | El cálculo tardó 0,003274 s sin escribir archivos (una sola medición). |
| Integración con otras herramientas | Con su extensión CSV pudo leer la serie que generó Java. | No asignado. | Copia solos los resultados y los parámetros a la carpeta de NetLogo. |

# 8. Análisis de Resultados

## 8.1. Evolución en Java (Figura 1)

Como se observa en la Figura 1, la corrida de Java empezó con 95 susceptibles, 5 infectados y ningún recuperado. Al revisar los datos, vi que al inicio el brote avanzó poco: en el paso 10 había 11 infectados. Después aumentaron hasta llegar a 29 en el paso 31, que fue el punto más alto de esta ejecución. Para entonces quedaban 22 susceptibles y ya se habían recuperado 49 personas. A partir de ahí los infectados fueron disminuyendo hasta llegar a cero en el paso 77.

Lo que más me llamó la atención fue que al final el brote avanzó muy lentamente. Entre los pasos 63 y 76 quedó una sola persona infectada y no hubo nuevos contagios. Esto puede ocurrir porque, con pocos susceptibles, es menos probable que el contacto elegido sea alguien que todavía pueda contagiarse; además, cada paso existe la posibilidad de que la persona infectada se recupere. Al terminar, quedaron 13 susceptibles y 87 recuperados. En otras palabras, 87 personas habían pasado por la infección, incluidos los cinco casos iniciales, mientras que 13 nunca se contagiaron.

## 8.2. Evolución en NetLogo (Figura 2)

En la Figura 2 se aprecia que NetLogo también se empezó con 95 susceptibles, 5 infectados y 0 recuperados, pero la evolución fue distinta. En el paso 10 ya había 36 infectados y el máximo fue de 44 en el paso 11, valor que se mantuvo en el paso 12. En comparación con Java, el crecimiento fue más rápido y el pico apareció antes. Luego el número

de infectados fue bajando hasta llegar a cero en el paso 55.

En esta corrida, los susceptibles llegaron a 8 en el paso 28 y ya no disminuyeron después de ese momento. Al final quedaron 8 susceptibles y 92 recuperados; por tanto, 92 personas habían pasado por la infección. Al comparar ambas ejecuciones, NetLogo tuvo un pico 15 personas mayor y la infección terminó 22 pasos antes que en Java. Es una diferencia visible en los datos, aunque por sí sola no significa que NetLogo propague más la enfermedad, solo que estamos comparando la ejecución de cada herramienta.

## 8.3. Comparación de susceptibles, infectados y recuperados (Figuras 3, 4 y 5)

**Susceptibles (Figura 3).** En la Figura 3 se ve que en las dos herramientas la cantidad de susceptibles empezó en 95, pero disminuyó de manera diferente. En Java quedaron 13 al final y en NetLogo quedaron 8. Esto significa que, en la corrida de NetLogo, cinco personas más se contagiaron. La diferencia se relaciona con cómo evolucionó cada ejecución, no con un cambio en los parámetros, que se mantuvieron iguales.

**Infectados (Figura 4).** La Figura 4 muestra la diferencia más evidente entre las curvas: Java llegó a 29 infectados en el paso 31, mientras que NetLogo llegó a 44 en el paso 11. También se observa que la infección desapareció antes en NetLogo: en el paso 55 ya no había infectados, mientras que en Java esto ocurrió en el paso 77. Aunque las reglas eran las mismas, los contagios y las recuperaciones ocurrieron en distinto orden debido al azar de cada simulación.

**Recuperados (Figura 5).** En la Figura 5, el número de recuperados fue aumentando a medida que avanzó cada brote. Al llegar al paso 100, Java registró 87 recuperados y NetLogo 92. Como en ambas ejecuciones ya no quedaban personas infectadas, estos valores indican cuántas personas pasaron por la infección, contando también a los cinco infectados iniciales. Representan el 87 % y el 92 % de la población, respectivamente.

## 8.4. ¿Por qué difieren dos implementaciones del mismo modelo?

Aunque usamos los mismos parámetros y la misma semilla 42, Java y NetLogo no generan la misma secuencia de números pseudoaleatorios. La semilla sirve para repetir una corrida dentro de cada herramienta, pero no obliga a que ambas elijan los mismos contactos ni obtengan los mismos resultados. También influye el orden en que se recorren las personas y se realizan los sorteos. Por eso, en nuestras ejecuciones los picos y la duración del brote fueron distintos.

Con estos datos podemos describir lo que ocurrió en cada corrida, pero no concluir que una herramienta produzca brotes más grandes o más cortos. Para comparar sus resultados con mayor seguridad habría que repetir la simulación con varias semillas y analizar medidas como el promedio y la variación de los picos y de la duración.

## 8.5. Efecto de la recuperación (Figura 6, Tabla 4)

En la Figura 6 se comparan las curvas de infectados para ambos valores de recuperación. Al aumentar la probabilidad de recuperación de $\gamma = 0{,}10$ a $\gamma = 0{,}40$, la relación $\mathcal{R}_0 = \beta/\gamma$ pasa de 3 a 0,75, manteniendo $\beta = 0{,}30$. En la corrida de Java con el nuevo valor, el número de infectados llegó a un máximo de 6 en el paso 1 y descendió a cero en el paso 9. El estado final fue $(S, I, R) = (89, 0, 11)$, por lo que el brote se detuvo pronto y solo 11 personas pasaron por la infección.

Estos resultados son coherentes con una recuperación más probable: las personas infectadas tienen más oportunidades de recuperarse antes de que se produzcan nuevos

contagios. La trayectoria de NetLogo debe compararse con sus propios datos; aunque coincida el estado final, eso no implica que ambas curvas hayan seguido el mismo camino.

### 8.6. Ventajas, limitaciones y ámbito de aplicación

Durante la práctica, NetLogo resultó más cómodo para observar el comportamiento del modelo porque permite representar a cada persona como un agente y ver los cambios de estado en pantalla, además de mostrar las gráficas. En Java hubo que escribir más código para manejar las personas, ejecutar las reglas y guardar los datos, pero esa organización permite controlar con detalle la lógica de la simulación y reutilizar sus resultados. No implementamos Python/SimPy, así que la comparación de esa herramienta se limita a lo consultado en su documentación y no a una experiencia de uso.

Una limitación del trabajo es que se hizo una sola corrida por herramienta para la comparación base. Como el modelo depende del azar, los resultados pueden variar entre ejecuciones. Además, los tiempos registrados no se midieron bajo condiciones equivalentes, por lo que no sirven para afirmar cuál herramienta es más rápida en general.

### 8.7. Alcance de la integración

Guardar los resultados de Java en archivos CSV permitió separar el cálculo de la visualización. El archivo de resultados contiene la evolución de susceptibles, infectados y recuperados, mientras que el archivo de parámetros deja constancia de los valores usados en la corrida. NetLogo puede leer esos archivos y mostrar la serie de Java sin volver a calcularla, lo que facilita revisar y comparar los datos.

La integración también tiene una limitación: no es automática en tiempo real. Primero se ejecuta Java para generar los archivos y después se actualizan los datos en NetLogo. Aun así, para esta práctica fue suficiente para comparar las curvas dentro de una misma interfaz.

## 9. Preguntas de Control

1. **¿Qué representa el modelo SIR?**

   Es un modelo que muestra cómo se propaga una enfermedad en un grupo de personas. Para eso reparte a la gente en tres estados (S, I y R) y va contando cuántas personas hay en cada uno a medida que pasa el tiempo [1]. Con eso se puede ver cuándo llega el pico del brote y cuándo se acaba. En la práctica lo simulamos persona por persona, con $N = 100$.

2. **¿Qué representan los estados S, I y R?**

   **S** son las personas susceptibles: no se han contagiado, pero pueden hacerlo. **I** son las infectadas: tienen la enfermedad y pueden contagiar a otras. **R** son las recuperadas: ya se curaron y no vuelven a contagiarse [1]. Como en nuestro modelo nadie nace, muere ni llega de afuera, siempre se cumple $S + I + R = 100$.

3. **¿Qué función cumplen los parámetros $\beta$ y $\gamma$?**

   $\beta$ es la probabilidad de contagio: qué tan fácil es que una persona infectada contagie a una susceptible (usamos $0{,}30$). $\gamma$ es la probabilidad de recuperarse en cada paso (usamos $0{,}10$), así que una persona tarda en promedio unos $1/\gamma = 10$ pasos en recuperarse [1]. Con los dos sale $\mathcal{R}_0 = \beta/\gamma = 3$ en la corrida base; con $\gamma = 0{,}40$ baja a $0{,}75$ y el brote se apaga rápido.

4. **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?**

Porque si cada herramienta usara valores distintos, no sabríamos si la diferencia en los resultados viene de la herramienta o de los parámetros. Con la misma población, los mismos infectados iniciales, el mismo $\beta$, el mismo $\gamma$ y las mismas reglas, lo único que cambia es la implementación y el azar [1]. Así la comparación es justa.

5. **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?**

   Porque el azar no es el mismo en cada herramienta. Los números "al azar" de un programa son pseudoaleatorios: salen de una fórmula que parte de una semilla. Java usa una fórmula lineal congruencial con una semilla de 48 bits [3], mientras que NetLogo usa el Mersenne Twister [2]. Poner la misma semilla (42) hace que cada herramienta se repita a sí misma, pero no hace que sorteen los mismos números. Además, en NetLogo los agentes se recorren en un orden aleatorio distinto cada vez [2], y el orden en que se hacen los sorteos cambia la historia. Por eso en nuestras corridas Java llegó a un pico de 29 infectados y NetLogo a uno de 44, con las mismas reglas. Con Python pasaría algo parecido. Igual, con una sola corrida por herramienta no podemos decir que una contagie más que la otra.

6. **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**

   Que ya trae en un solo lugar todo lo que necesitamos: agentes (tortugas) con estado y color, el mundo donde se ven, botones, monitores y gráficas [2]. Eso nos permitió ver el brote moviéndose y cambiando de color sin programar una interfaz aparte. También es fácil cambiar una regla y ver al instante qué pasa con los agentes. En general, los modelos basados en agentes sirven para ver cómo unas reglas sencillas a nivel individual producen un resultado en todo el grupo [6].

7. **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**

   Esta respuesta es solo conceptual, porque no implementamos SimPy. Según su documentación, es una biblioteca de Python para simulación de eventos discretos, donde los procesos se escriben como funciones generadoras y se pueden usar recursos compartidos para representar colas o servidores [5]. Además, la simulación puede correr lo más rápido posible, en tiempo real o paso a paso [5]. Eso le conviene a sistemas con eventos y esperas (clientes, máquinas, turnos). Para nuestro SIR, que avanza en pasos fijos, la misma documentación dice que SimPy sería excesivo cuando los procesos no interactúan entre sí ni con recursos compartidos [5].

8. **¿Qué ventajas ofrece Java para implementar modelos de simulación?**

   Da control total del código. Pudimos representar a cada persona como un objeto con un estado enumerado, separar el programa en modelo, vista y controlador, y usar `Random` con semilla para repetir la corrida [3]. Además, solo usamos la biblioteca estándar, sin librerías externas, y escribir los resultados a CSV fue sencillo. Lo malo es que hay que escribir más código que en NetLogo, y la visualización no viene incluida: por eso usamos NetLogo para ver las curvas. Para el tiempo, tenemos una sola medición de $0{,}003274$ s para calcular la simulación (sin escribir archivos), que no alcanza para comparar rendimiento.

9. **¿Qué función cumple el archivo CSV en la integración Python/Java $\rightarrow$ NetLogo?**

   Es el puente entre las dos herramientas. Un CSV es un archivo de texto con datos separados por comas que otros programas pueden leer fácilmente [4]. Java guarda ahí la serie de S, I y R (resultados_java.csv) y los parámetros usados (parametros_java.csv). NetLogo los lee con su extensión `csv`, que puede cargar un archivo

completo como una lista de listas [4], y con eso dibuja la curva de Java sin recalcularla ni modificarla.

10. **¿Qué ventajas presenta separar la generación de datos de la visualización?**

    Primero, el cálculo y la visualización no dependen uno del otro: podemos cambiar las gráficas sin tocar la simulación, o cambiar el modelo sin rehacer la interfaz. Segundo, el cálculo se hace una vez y se puede visualizar muchas veces. Por ejemplo, para probar $\gamma = 0{,}40$ solo hubo que volver a correr Java y pulsar ACTUALIZAR JAVA en NetLogo. Tercero, los archivos guardan los resultados y los parámetros usados, lo que ayuda a repetir y revisar lo que se hizo, una idea que también se defiende en las guías de análisis reproducible para simulación [7]. La limitación es que el flujo va por archivos: primero se corre Java y después se actualiza NetLogo.

# 10. Conclusiones

- Implementar el modelo SIR en Java y NetLogo me permitió comprobar que se puede trabajar con las mismas reglas y parámetros y, aun así, obtener curvas distintas. En las dos corridas se conservaron los 100 individuos y se guardaron 101 registros, desde $t = 0$ hasta $t = 100$. Python/SimPy no se implementó porque no fue la herramienta asignada al grupo.
- En mi experiencia, NetLogo fue más práctico para observar el brote, ya que muestra a las personas y sus cambios de estado directamente. Java requirió más código para representar a las personas y guardar los resultados, pero me permitió organizar el cálculo y exportar los datos a CSV.
- En la corrida base, Java alcanzó 29 infectados en el paso 31 y NetLogo llegó a 44 en el paso 11. La diferencia se debe a la evolución aleatoria de cada ejecución; con una sola corrida por herramienta no se puede afirmar que una produzca brotes mayores de forma general. Para sostener esa comparación habría que repetir las simulaciones con distintas semillas.
- Al subir $\gamma$ a $0{,}40$ en Java, el máximo fue de 6 infectados y la infección llegó a cero en el paso 9. Además, usar CSV facilitó mostrar los datos de Java en NetLogo sin volver a calcularlos allí, aunque fue necesario ejecutar Java y actualizar la lectura de los archivos manualmente.

# 11. Declaración de uso de Inteligencia Artificial

En esta práctica utilicé herramientas de inteligencia artificial para dos cosas: Claude me sirvió como apoyo para construir el código del modelo en NetLogo. Y por otro lado, utilicé NotebookLM para trabajar las respuestas de las preguntas de control y preparar el archivo de bibliografía referencias.bib.

## 11.1. Prompts utilizados

**Prompt 1: Claude (código de NetLogo).**

> *Estoy trabajando en una práctica de simulación de un modelo SIR con 100 personas. Ayúdame a construir el código para el lenguaje de NetLogo, pues no soy muy experto en él. siguiendo las reglas y los parámetros indicados en mi codigo ya implementado en Java*

**Prompt 2: NotebookLM (preguntas de control y bibliografía).**

> *Usando únicamente las fuentes que he cargado sobre la práctica SIR y la documentación de las herramientas, ayúdame a responder las preguntas de control con explicaciones claras y directas. Señala qué información respalda cada respuesta y*

*conserva las referencias para poder citarlas en el informe. Después, organiza los datos bibliográficos de las fuentes que realmente se utilizaron en formato.bib sin inventar autores, títulos, fechas ni enlaces, tambien indica si falta algún dato para completar una referencia para yo consultarlo.*

### 11.2. Justificación del uso

**Herramientas utilizadas.**

- **Claude:** apoyo para construir el código del modelo en NetLogo.
- **NotebookLM:** apoyo para elaborar las respuestas a las preguntas de control y preparar el archivo referencias.bib a partir de las fuentes consultadas.

**Trabajo realizado sin IA.**

- Elaboración del código en JAVA, la obtención de los resultados numéricos y la elaboración de las gráficas se realizaron a partir del código y de los datos de las corridas.
- La redacción del procedimiento, la metodología, el análisis de resultados y las conclusiones fue realizada por mí, tomando como base el trabajo efectuado y los resultados observados.

## 12. Bibliografía

[1] J. O. Guamán Q., “Guía de actividades práctico-experimentales nro. 004: Análisis comparativo de herramientas de simulación mediante un modelo común.” FEIRNNR, Carrera de Computación, Universidad Nacional de Loja. Documento proporcionado por el docente, 2026.

[2] Center for Connected Learning and Computer-Based Modeling, “Programming guide.” NetLogo 7.0.4 User Manual. [En línea]. Disponible en: https://docs.netlogo.org/7.0.4/programming. [Consulta: 25 de octubre de 2026].

[3] Oracle, “Class Random.” Java Platform, Standard Edition 11 API Specification. [En línea]. Disponible en: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Random.html. [Consulta: 25 de octubre de 2026].

[4] Center for Connected Learning and Computer-Based Modeling, “CSV extension.” NetLogo 7.0.4 User Manual. [En línea]. Disponible en: https://docs.netlogo.org/7.0.4/csv. [Consulta: 25 de octubre de 2026].

[5] Team SimPy, “SimPy: Discrete event simulation for Python.” SimPy 4.1.2 documentation (Overview). [En línea]. Disponible en: https://simpy.readthedocs.io/en/stable/. [Consulta: 25 de octubre de 2026].

[6] I. Romanowska, C. D. Wren, and S. A. Crabtree, *Agent-Based Modeling for Archaeology: Simulating the Complexity of Societies*. Santa Fe, NM, USA: The SFI Press, 2021.

[7] A. Heather, T. Monks, A. Harper, F. Alidoost, R. Challen, T. Slater, and N. Mustafee, “Reproducible analytical pipelines for healthcare discrete-event simulation: An open guide and worked examples,” *NIHR Open Research*, vol. 6, pp. 1–19, June 2026.

## 13. Anexos

### 13.1. Repositorio del proyecto

https://github.com/C-ael/Simulacion_Portafolio

## 13.2. Video de demostración

https://drive.google.com/file/d/1hf47qRfb9HUquimdc0U0JU-OOOfo_iox/view?usp=sharing